The cool thing about methods is that you can redefine them. So, here's your challenge: break the rules of math by redefining Fixnum#+ so that it always returns the correct result plus one. For example:

```
1 + 1    # => 3
```

Quiz Solution

You can solve this quiz with an *Open Class (14)*. Just reopen Fixnum and redefine + so that (x + y) becomes (x + y + 1). This is not as easy as it seems, however. The new version of + relies on the old version of +, so you need to wrap your old version with the new version. You can do that with an *Around Alias (134)*:

class_definitions/broken_math.rb

```
class Fixnum
  alias_method :old_plus, :+

  def +(value)
    self.old_plus(value).old_plus(1)
  end
end
```

Now you have the power to wreak havoc on Ruby's basic arithmetic. Enjoy this code responsibly.

Wrap-Up

You covered a lot of ground today. Let's sum it all up:

- You looked at the effects of class definitions on self (the default receiver of the methods you call) and on the current class (the default home of the methods you define).
- You made acquaintance with *Singleton Methods (114)* and singleton classes, gaining new insights into the object model and method lookup.
- You added a few new tricks to your bag, including *Class Instance Variables (109)*, *Class Macros (117)*, and *Prepended Wrappers (136)*.

Also remember that today you used the word "class" as a shortcut to "class or module," and everything you learned about classes can also be applied to modules: the "current class" might actually be a module, a "class instance variable" could well be a "module instance variable," and so on.

That was quite a deep dive into Ruby's object model. As the two of you prepare to leave the office, Bill makes a promise that tomorrow will be less talking and more coding. "I'm really looking forward to that," he says.